PROCEDURAL KNOWLEDGE GENERATION WITH SELECTIVE RETRIEVAL FROM PAST OUTPUTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Retrieval-augmented generation (RAG) (Izacard et al., 2022) has become a common approach to ground the output of large language models (LLMs). For topics unseen in training, RAG requires careful search tuning to ensure the model sees the information necessary to answer the question, especially if it is dispersed across many documents. If we instead answer requests one at a time and store the responses, some information for later outputs will likely be densely available in earlier responses. We propose a new augmentation technique called Selective Retrieval from Past Outputs (SRPO) which searches a library of previous responses using model-generated queries, and filters them by having the model decide if each document is relevant. To demonstrate that this approach more easily surfaces useful information, we publish a new dataset called LCStep containing 120 clean, step-by-step procedures extracted from the LangChain Python documentation (created after the OpenAI knowledge cutoff date), along with a model-based evaluation metric that judges the quality of generated procedures. We show that GPT-4 cannot reliably generate these procedures from a candidate goal, even when augmented with retrieved texts from LangChain's conceptual documentation and API reference. We demonstrate that collecting outputs into a library of retrievable procedures can improve later responses, as long as retrieved procedures are highly relevant to the current generation.

1 LCSTEP

LCStep contains three sets of documents: API reference, conceptual documentation, and procedures. See Figure 1 for a diagram of the process of generating the LCStep data.

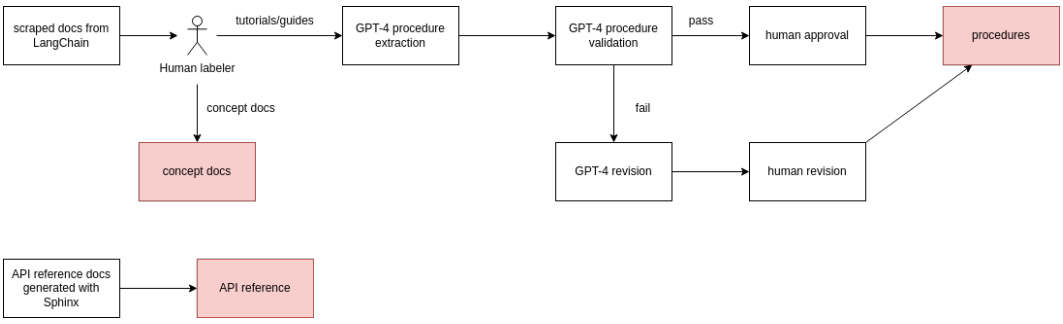

Figure 1: The workflow used to generate the LCStep dataset.

1.1 API REFERENCE

We generate the API reference material from the source files in the LangChain GitHub repository[1] using Sphinx. These files contain descriptions of all APIs in the Python package, including call signatures and argument descriptions. These files do not contain any usage examples or high-level explanation.

[1] https://github.com/langchain-ai/langchain/

1.2 CONCEPTUAL DOCUMENTATION AND PROCEDURES

We collect these documents by scraping the LangChain Python docs[2]. We manually filter out topic pages and stubs, leaving 228 documents. We then manually classified these into 137 documents of conceptual documentation, and 91 documents containing tutorials/guides.

For the 91 tutorials/guides, we prompted GPT-4[3] to extract a list of high-level steps necessary to accomplish the goal. We then prompted GPT-4[4] to rate those extracted procedures using a list of criteria. We found that this caught many mistakes where GPT-4 did not follow all the stated instructions. In those cases, we had the model revise the steps to meet the requirements, and then we manually checked the revised versions.

2 METHODS

Selective retrieval from past outputs (SRPO) involves two parts: selecting candidate documents from source material and past generations, and then the selection and refinement of those documents.

2.1 RETRIEVAL FROM PAST OUTPUTS

The performance of retrieval-augmented generation (RAG) is highly dependent on the number and size of the documents, as well as the density of relevant information (citation needed). To reliably discover the information the model needs, it is already established practice to threshhold retrieved documents based on embedding similarity with the query[5], or to prompt an LLM to filter[6] or compress[7] documents to include just the relevant information in the final prompt to answer the query.

For many tasks, a lot of the knowledge to be retrieved is the same for many queries. In our case, many procedures involving the LangChain library require connecting to a language model, creating a prompt template, and constructing a chain. Instead of collecting this information anew for each similar query, selective retrieval from past outputs (SRPO) searches over past outputs to find procedures similar to the current query, and then includes them in the set of retrieved candidate documents to include in the final prompt.

2.2 SELECTION AND REFINEMENT

Once we have a set of candidate documents, we apply similar filtering/compression techniques as is common, but we encourage the model to

3 EXPERIMENTS

To demonstrate that SRPO can overcome shortcomings of standard retrieval, we pit the two methods against each other on the task of generating the procedures in LCStep. We also test against a few baselines and oracles which we explain below. Both methods have access to the API reference and conceptual documentation from the dataset.

[2] https://python.langchain.com/

[3] See Appendix A.1 for the full prompt.

[4] See Appendix A.2 for the full prompt.

[5] `https://python.langchain.com/docs/modules/data_connection/retrievers/contextual_compression/#embeddingsfilter`

[6] `https://python.langchain.com/docs/modules/data_connection/retrievers/contextual_compression/#llmchainfilter`

[7] `https://python.langchain.com/docs/modules/data_connection/retrievers/contextual_compression/#adding-contextual-compression-with-an-llmchainextractor`. Gilbert et al. (2023) study GPT-4's ability to compress texts, but ask the model to remove extraneous information. Should we include a study of the value of contextual compression as described in that LangChain doc? I can't find any papers that look into this.

3.1 BASELINES

All prompting methods are performed with `GPT-4-0614` from OpenAI as well as a version of LLaMA2 instruction-tuned by Thales.

- The **few-shot** approach will prompt the model with a basic instruction and a few fixed examples of the output format. We expect this method to fail due to a lack of knowledge of LangChain in the models' training data.
- The vanilla retrieval-augmented approach (**RAG**) constructs a prompt by doing retrieval over a vector store containing the conceptual documentation and API reference material.
  - We expect to further refine this retrieval process somewhat. This may involve enforcing retrieval diversity by penalizing similarity between retrieved docs, having the model split the query into subqueries to find the individual parts necessary to build the whole answer, or using contextual compression[8].
  - This method would also benefit from HyDE (Gao et al., 2022), I^3 (Dong et al., 2023), or EAR (Chuang et al., 2023). At that point, it might be worth having this optimized version as a separate baseline here.

For our experiment, the **SRPO** approach constructs a prompt by retrieval over the same documents as RAG, plus all procedures generated so far. When retrieving from the past outputs, the model is separately presented with the selection prompt as described in Section 2.

3.2 EVALUATION

Try to connect to metrics already in use in other papers, like ones for truthfulness and completeness (see page 39 of https://arxiv.org/pdf/2307.10928.pdf)

https://github.com/X-PLUG/mPLUG-Owl/blob/main/OwlEval/OwlEval.md

TODO "model card" for this paper: write down numbers: dataset size, approaches, models, evaluations, etc. - we need a fine-tuned baseline - another task: question-answering

RAG has been very limited so far but now input tokens are getting longer, is there a way to involve

how do we draw the line between using RAG and fine-tuning, based on the size and label types and publicness of your dataset. So we want lots of comparisons for this reason too. Set smaller token limits artificially for 100k models, and do the same for llama2 (the OSS standard).

https://huggingface.co/docs/peft/index https://arxiv.org/pdf/2307.10928.pdf

REFERENCES

Yung-Sung Chuang, Wei Fang, Shang-Wen Li, Wen tau Yih, and James R. Glass. Expand, rerank, and retrieve: Query reranking for open-domain question answering. *Annual Meeting Of The Association For Computational Linguistics*, 2023. doi: 10.48550/arXiv.2305.17080.

Qian Dong, Yiding Liu, Qingyao Ai, Haitao Li, Shuaiqiang Wang, Yiqun Liu, Dawei Yin, and Shaoping Ma. I^3 retriever: Incorporating implicit interaction in pre-trained language models for passage retrieval. *arXiv preprint arXiv: 2306.02371*, 2023.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. *Annual Meeting Of The Association For Computational Linguistics*, 2022. doi: 10.48550/arXiv.2212.10496.

Henry Gilbert, Michael Sandborn, Douglas C. Schmidt, Jesse Spencer-Smith, and Jules White. Semantic compression with large language models. *arXiv preprint arXiv: 2304.12512*, 2023.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv: 2208.03299*, 2022.

[8] `https://python.langchain.com/docs/modules/data_connection/retrievers/contextual_compression/`

A LCSTEP PROMPTS

A.1 INSTRUCTIONS FOR FORMATTING PROCEDURES

You are helping format procedures. I will give you raw procedure text, describing how to complete a procedure, with code and examples. Your task is to simplify and generalize this procedure to achieve the desired format. The desired format is: a title, description, a minimal set of simple necessary instructions for general application where every instruction is describing one action and the API references to use in that action (with the module the API reference belongs to), and side notes if needed. Remove all unnecessary details, code, examples and specific cases. Keep only the strictly necessary steps to accomplish the procedure.The procedure should start with a title and a brief description containing any important information that doesn't fit into the instructions. Do not mention importing the required modules as a separate step, as we will include the required imports from the API references made in the instructions. If there are any checks, or case-specific instructions, they should be specified as a side note after the instructions as we want the procedure instructions to be for general use. If an API reference has parameters, make sure to mention these parameters in the instruction. If the raw text is actually describing multiple different procedures, you may then have the formatted procedure split into multiple different sets of instructions for each different procedure, but this should only be done if judged necessary.

A.2 INSTRUCTIONS FOR VALIDATING PROCEDURES

TODO